SOFTWARE REQUIREMENTS SPECIFICATION

AI Mood-Based Movie Recommendation System

MCA Final Year Project

A web-based intelligent movie recommendation platform that analyzes a user's mood and recommends suitable movies with relevant movie information.

Technology: React.js, Node.js, Express.js, MongoDB, Movie API, AI/NLP

Prepared for MCA Final Year Project

1. Introduction

The AI Mood-Based Movie Recommendation System is a web application designed to recommend movies according to the user's current mood. The user can select a mood such as Happy, Sad, Romantic, Angry, Excited, Relaxed or Motivated, or enter a short sentence describing how they feel. The system processes this input and recommends relevant movies.

Each recommendation provides the movie name, release date, IMDb rating, genre, poster and a short overview. The system can also provide watchlist, favorites, ratings and reviews to improve personalization.

1.1 Purpose

The purpose of this project is to provide an intelligent and user-friendly movie discovery platform that reduces the effort required to find a movie suitable for the user's current emotional state.

1.2 Scope

• User registration and secure login.

• Mood selection and optional text-based mood detection.

• AI-assisted mood classification.

• Movie recommendation based on mood, genre and user preferences.

• Movie details including title, release date, IMDb rating and short description.

• Movie search and filtering.

• Watchlist and favorites.

• User ratings and reviews.

• Admin management and basic analytics.

2. Overall Description

2.1 Product Perspective

The system follows a client-server architecture. The React frontend communicates with a Node.js/Express backend through REST APIs. MongoDB stores user, review, watchlist and application data. A movie data service such as TMDb or OMDb can provide movie metadata, while an AI/NLP service can analyze free-text mood input.

2.2 User Classes

• Normal User: registers, logs in, selects mood, receives recommendations, views movie details, manages watchlist and submits reviews.

• Administrator: manages users, movies/recommendation data, reviews and application analytics.

2.3 Operating Environment

• Client: Modern web browser such as Chrome, Edge or Firefox.

• Frontend: React.js with responsive CSS/Tailwind CSS.

• Backend: Node.js with Express.js.

• Database: MongoDB.

• Development OS: Windows, Linux or macOS.

• Internet connection required for external movie/AI APIs.

2.4 Assumptions and Dependencies

• The external movie API is available and its usage limits are respected.

• AI/NLP service availability and API credentials are maintained.

• Users provide valid information during registration.

• IMDb rating information is displayed only when available from the selected movie data source.

3. Functional Requirements

ID	Requirement
FR-01	The system shall allow users to create an account.
FR-02	The system shall authenticate registered users securely.
FR-03	The system shall allow a user to select a mood.
FR-04	The system shall accept optional natural-language mood input.
FR-05	The system shall classify text into a supported mood category.
FR-06	The system shall generate movie recommendations based on mood and available movie metadata.
FR-07	The system shall display movie title, release date, rating and short overview.
FR-08	The system shall allow movie search.
FR-09	The system shall allow users to add or remove movies from a watchlist.
FR-10	The system shall allow users to submit ratings and reviews.
FR-11	The system shall maintain recommendation/history information where enabled.
FR-12	The admin shall be able to manage users and reviews.

4. Non-Functional Requirements

• Performance: common pages should load quickly and API requests should be handled efficiently.

• Security: passwords shall be hashed; authenticated APIs shall use secure tokens and authorization checks.

• Usability: the interface shall be simple, responsive and easy for beginner users.

• Reliability: API failures should be handled gracefully with meaningful error messages.

• Scalability: the backend should be organized so additional recommendation methods can be added later.

• Maintainability: frontend, backend, services and database models should be separated into logical modules.

• Compatibility: the application should work on modern desktop and mobile browsers.

5. System Features

5.1 Authentication

Registration, login, logout and protected routes. JWT can be used for authentication and bcrypt for password hashing.

5.2 Mood Detection

The user may choose a predefined mood or enter natural-language text. Example: 'I am feeling low today and want something uplifting.' The AI/NLP component classifies the input into a suitable mood category.

5.3 Recommendation Engine

The recommendation engine maps mood categories to relevant genres, themes and movie metadata. It can combine mood relevance, movie rating, popularity, genre and user preferences to rank recommendations.

5.4 Movie Details

The details page shall display the movie poster, title, release date, IMDb rating when available, genre, runtime when available, short overview, cast/director when available and trailer/external information where supported.

5.5 Search and Filters

Users can search by movie title and optionally filter by genre, year, rating or language depending on the movie API data.

5.6 Watchlist and Reviews

Users can save movies for later and submit ratings/reviews. Reviews can be used as optional input for future personalization and sentiment analysis.

5.7 Admin Panel

Administrators can manage application users, moderate reviews and view basic usage statistics.

6. AI Recommendation Logic

A simple first version can use a hybrid approach. Mood is mapped to preferred genres, and candidate movies are ranked using mood relevance plus rating/popularity. Later versions can incorporate user history.

• Happy → Comedy, Family, Adventure, feel-good movies.

• Sad → Drama, Inspirational, Feel-good movies.

• Romantic → Romance, Drama, Romantic Comedy.

• Angry → Action, Thriller, Adventure.

• Excited → Action, Adventure, Sci-Fi.

• Relaxed → Comedy, Family, Animation, Light Drama.

• Fearful → Horror, Mystery, Thriller.

• Motivated → Biography, Sports, Inspirational, Drama.

These mappings are initial project rules and can be improved after testing with real user feedback.

7. System Architecture

Presentation Layer: React.js, Tailwind CSS and React Router.

Application Layer: Node.js and Express.js REST APIs.

AI Layer: mood classification and recommendation logic.

Data Layer: MongoDB.

External Services: movie metadata API and optional AI service.

8. Database Requirements

8.1 Users Collection

Fields: _id, name, email, passwordHash, role, favoriteMovies, watchlist, createdAt.

8.2 Movies Collection

Fields: _id, externalMovieId, title, releaseDate, genres, overview, posterUrl, rating, runtime, language.

8.3 Reviews Collection

Fields: _id, userId, movieId, rating, comment, sentiment, createdAt.

8.4 Mood History Collection

Fields: _id, userId, inputText, detectedMood, recommendationIds, createdAt.

9. API Requirements

• POST /api/auth/register — register user.

• POST /api/auth/login — authenticate user.

• GET /api/movies/search — search movies.

• GET /api/movies/:id — get movie details.

• POST /api/mood/analyze — analyze mood text.

• POST /api/recommendations — generate recommendations.

• GET /api/watchlist — get user's watchlist.

• POST /api/watchlist — add a movie.

• DELETE /api/watchlist/:id — remove a movie.

• POST /api/reviews — create a review.

• GET /api/reviews/:movieId — get movie reviews.

10. Use Case Summary

Actor	Use Cases
User	Register, Login, Select Mood, Enter Mood Text, Get Recommendations, Search Movie, View Details, Add to Wa
Admin	Login, Manage Users, Moderate Reviews, Manage Movie Data, View Analytics

11. User Interface Requirements

• Responsive landing/home page.

• Clean login and registration forms.

• Mood cards with icons and labels.

• Recommendation cards showing poster, title and rating.

• Movie details page with clear metadata and overview.

• Watchlist page.

• Profile page.

• Admin dashboard with tables/cards/charts.

12. Security Requirements

• Passwords must never be stored as plain text.

• JWT tokens must be validated for protected endpoints.

• Role-based authorization shall protect admin functions.

• Input validation shall be applied to forms and APIs.

• External API keys shall be stored in environment variables, not frontend source code.

13. Testing Requirements

• Unit testing of recommendation and utility functions.

• API testing for authentication, movies, mood and watchlist endpoints.

• Integration testing between frontend, backend and database.

• Validation testing for invalid login, empty mood input and API failures.

• Responsive UI testing on desktop and mobile screen sizes.

• User acceptance testing with sample users and different moods.

14. Development Plan

Phase	Major Work
1	Requirement analysis, SRS, UI wireframes and database design.
2	React setup, routing and common UI components.
3	Login, registration and protected routes.
4	Node/Express backend and MongoDB integration.
5	Movie API integration and movie details.
6	Mood detection and recommendation engine.
7	Watchlist, reviews, profile and admin panel.
8	Testing, bug fixing, deployment and final documentation.

15. Future Enhancements

• Voice-based movie search.

• Multilingual mood input.

• More advanced collaborative filtering.

• Deep-learning-based emotion classification.

• Personalized recommendations using long-term watch history.

• Mobile application version.

• Streaming-platform availability integration where legally and technically supported.

16. Conclusion

The AI Mood-Based Movie Recommendation System provides a practical MCA-level application combining full-stack web development, external API integration and AI-assisted personalization. The proposed SRS defines the major functional, non-functional, data, API, security and testing requirements needed to implement the system in a structured manner.